

MASTER OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

Mitigating Free-Riding Behavior in Federated Learning Systems

Aashnan Rahman

231041006

Department of Computer Science and Engineering

Islamic University of Technology

July, 2026

Mitigating Free-Riding Behavior in Federated Learning Systems

Aashnan Rahman

231041006

Department of Computer Science and Engineering

Islamic University of Technology

July, 2026

Certificate of Approval

The thesis titled, *Mitigating Free-Riding Behavior in Federated Learning Systems* submitted by Aashnan Rahman with student ID 231041006 of Academic Year 2024–25 has been found as satisfactory and accepted as partial fulfillment of the requirement for the degree Master of Science in Computer Science and Engineering on July 04, 2026.

Board of Examiners:

Dr. DJ Ikstra

Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Chairman
(Supervisor)

Dr. Mike Hunt

Professor

Department of Electrical and Computer Engineering

Imaginary University of Technology, Texas, US

Member
(Co-Supervisor)

Dr. Justin Case

Professor and Head and Head

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member
(Ex-Officio)

Dr. Benjamin Dover

Assistant Professor and Head

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member

Member

Mr. Noah Pinion

Assistant Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur, Bangladesh

Member
(External)

Toby Lerone

Professor and Head

Department of Computer Science and Engineering

Bjork University, UK

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by Aashnan Rahman under the supervision of Dr. DJ Ikstra, Professor, Department of Computer Science and Engineering, Islamic University of Technology (IUT), Dhaka, Bangladesh and co-supervision of Dr. Mike Hunt, Professor, Department of Electrical and Computer Engineering, Imaginary University of Technology, Texas, US . It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Dr. DJ Ikstra

Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT), Gazipur,

Bangladesh

Date: July 04, 2026

Aashnan Rahman

Student ID: 231041006

Date: July 04, 2026

Dr. Mike Hunt

Professor

Department of Electrical and Computer Engineering

Imaginary University of Technology, Texas, US

Date: July 04, 2026

Dedicated to To be added later

Contents

1	Introduction	1
1.1	Introduction to Federated Learning and the Free Rider Problem	1
1.2	Scope and Motivation	2
1.3	Problem Statement	3
1.4	Research Challenges	4
1.5	Contributions	6
1.5.1	Organization	7
2	Related Work	8
2.1	Background	8
2.2	Foundational Characterization of Free-Rider Attacks	8
2.3	Advanced and Stealthy Free-Rider Attacks	9
2.4	Server-Side Detection and Audit-Based Mitigation	10
2.5	Privacy-Attack-Based Detection	11
2.6	Hardware-Based and Cryptographic Solutions	13
2.7	Other Notable Approaches	13
2.8	Summary and Research Gap	14
3	Proposed Methodology	16
3.1	Overview	16
3.2	Notation	17
3.3	Threat Model	17
3.4	Framework Description	18
3.4.1	Stage 0 – Warm-Up Phase (Rounds 1–10)	18
3.4.2	Stage 1 – Coverage Sweep Phase	18
3.4.3	Stage 2 – Suspicion-Weighted Sweep Phase (Ongoing)	18
3.4.4	Stage 3 – Reference Computation for Suspicion-Weighted Subsets	19
3.4.5	Stage 4 – Trap Model Broadcast and Local Training	19

3.4.6	Stage 5 – Zero-Update Check (Every Round)	20
3.4.7	Stage 6 – Cosine Similarity Check (Every Round)	20
3.4.8	Stage 7 – Aggregation (Subset-Scoped Discard)	21
3.4.9	Stage 8 – Bookkeeping and Reset Trigger	21
3.4.10	Stage 9 – Removal	21
3.5	Algorithm	21
3.6	Datasets and Data Distribution	21
3.7	Experimental Design	23
3.8	Evaluation Metrics	23
3.9	Design Rationale	24
3.10	Limitations and Open Problems	25
4	Results and Discussion	27
5	Conclusion	28
	References	29
	Appendices	32
A	Some Appendix Chapter	33
B	Some Other Appendix Chapter	34

List of Figures

A.1 A figure in Appendix	33
------------------------------------	----

List of Tables

2.1	Comparative summary of existing free-rider mitigation approaches . .	14
3.1	Summary of notation	17
3.2	Experimental configuration	23
B.1	A table in the appendix	34

List of Abbreviations

CNN	Convolutional Neural Network
PIP	PIP Installs Packages
TikZ	TikZ ist kein Zeichenprogramm
WIKI	What I Know Is

Acknowledgement

I am profoundly grateful to my supervisor, Dr. Lou Natic, for his endless patience, insightful critiques, and for always knowing just when to say, “Have you tried turning it off and on again?” His expertise and encouragement were instrumental in the completion of this thesis.

A special thanks to my co-supervisor, Dr. Sue Permann, who provided invaluable support and the occasional pep talk disguised as a lecture on time management. Her unwavering belief in my work kept me going, even when the only thing I believed in was the power of procrastination.

Finally, to my parents, whose love and support have been a constant source of strength. Thank you for always being there, for pretending to understand my research when I rambled on, and for not asking too many questions about why I was still not employed after all these years.

To all of you, I extend my heartfelt appreciation and a promise: next time, I will take fewer detours through the land of streaming services.

Abstract

Abstracts are crucial summaries of a thesis, balancing brevity and clarity while effectively conveying the study's essence. Despite their importance, many writers struggle with creating abstracts that are neither too vague nor overly detailed. This abstract tackles the challenge by offering a structured approach to abstract writing, breaking down the process into five essential components: Introduction, Problem, Proposed Solution, Results, and Conclusion. Using humorous examples, the abstract makes the abstract-writing process engaging and accessible. Our method demonstrates significant improvements in the quality and readability of abstracts, as the humor helps demystify and simplify the process. Ultimately, effective abstract writing is both an art and a science, requiring precision, audience awareness, and a touch of fun to make it approachable and enjoyable.

Chapter 1

Introduction

1.1 Introduction to Federated Learning and the Free Rider Problem

Federated Learning (FL) has emerged as a transformative paradigm for distributed machine learning that enables multiple clients such as mobile devices, edge servers, or organizational silos to collaboratively train a shared global model without exposing their raw, locally stored data [7], [11]. Unlike traditional centralized machine learning, where data must be aggregated on a single server, FL operates on a fundamentally different principle: *the model travels to the data, not the other way around*.

In a typical FL architecture, a central server initializes a global model and distributes it to a set of participating clients. Each client independently trains the model on its local dataset for a fixed number of epochs and computes a local model update (e.g., gradients or updated weights). These updates are transmitted back to the server, which aggregates them commonly via the Federated Averaging (FedAvg) algorithm to produce an improved global model [11]. This process is repeated over multiple communication rounds until the global model converges to a satisfactory performance level.

This architecture offers several compelling advantages:

- **Privacy Preservation:** Since raw data never leaves the client device, FL inherently reduces the risk of data leakage and aligns with data protection regulations such as GDPR and HIPAA.
- **Reduced Communication Overhead:** Only model parameters or gradients are exchanged, rather than entire datasets, which is particularly advantageous

in bandwidth-constrained environments.

- **Decentralized Data Utilization:** FL enables organizations and individuals to benefit from collaborative model training without relinquishing control over sensitive data.
- **Scalability:** FL systems can, in principle, incorporate thousands to millions of participating clients, as seen in large-scale mobile deployments such as Google’s keyboard prediction systems [2], [5].

Privacy is not merely a desirable feature of FL it is often the primary motivation for its adoption. In domains such as healthcare, finance, and personal mobile applications, the ability to train robust models while keeping sensitive data on-device is a critical enabler of machine learning adoption in regulated and privacy-sensitive contexts.

However, the very decentralization and privacy guarantees that make FL attractive also introduce a unique vulnerability: the server has limited visibility into what each client is actually doing locally. This opacity creates an opportunity for malicious or self-interested clients to exploit the system through what is known as **Free Rider (FR) behavior**. A free rider is a participant that contributes little or no genuine computational effort toward local model training, yet still submits an update to the server in order to receive the benefits of the collaboratively trained global model effectively benefiting from the effort of honest participants while incurring minimal cost. Because free riders can fabricate or manipulate updates that superficially resemble legitimate contributions, they are often difficult to distinguish from genuine, well-behaved clients using standard aggregation mechanisms alone.

1.2 Scope and Motivation

To understand why free rider behavior poses a serious threat to FL systems, consider a simple real-world analogy. Suppose a group project involves ten students who are collectively assigned a task, and their final grade depends on the group’s combined output. If six students genuinely contribute their effort while the remaining four merely submit superficial or copied work without doing any real analysis, the final result may still appear complete and acceptable to an external evaluator who only inspects the submitted output. Yet the four free-riding students receive the same reward (the grade) as the diligent six, despite having contributed little of value. Over time, this dynamic discourages honest participants, degrades the overall quality of the group’s output, and undermines the fairness of the collaborative process.

This scenario maps directly onto FL systems. Free-riding clients can submit updates that are statistically or structurally similar to genuine updates—for instance, by adding small perturbations to the previous global model, reusing outdated updates, or applying minimal or synthetic training without performing meaningful local training on their own data. Because the FL server typically aggregates updates based on volume or simple heuristics (e.g., number of local samples reported by the client) rather than verifying the training process itself, such free riders can successfully “blend in” with genuine contributors. They benefit from the improved global model at every round while incurring negligible computational, energy, or data costs.

The consequences of unmitigated free-rider behavior in FL systems are significant:

- **Degraded Model Quality:** A high proportion of free riders can slow convergence or degrade the accuracy of the global model, especially in non-IID data settings where genuine diverse contributions are essential.
- **Unfair Resource Allocation:** In incentive-based FL systems, free riders may unfairly claim rewards (monetary or otherwise) intended to compensate genuine contributors, discouraging honest participation over time.
- **Security and Trust Erosion:** Persistent free riding undermines trust in the FL ecosystem, potentially disincentivizing high-quality clients from participating altogether.
- **Scalability of Malicious Behavior:** As FL systems scale to include thousands of clients, manual or centralized verification of each client’s genuine contribution becomes computationally and practically infeasible, making the free rider problem increasingly urgent.

Given the growing deployment of FL in real-world, large-scale, and often adversarial environments, addressing free rider behavior is not merely an academic exercise but a practical necessity for ensuring the long-term viability, fairness, and robustness of federated systems.

1.3 Problem Statement

The core problem addressed in this thesis can be formally framed as follows:

In a Federated Learning system involving N clients participating over T communication rounds, a subset of clients $\mathcal{F} \subset \mathcal{N}$ may behave as free riders submitting model updates that do not reflect genuine training on locally avail-

able data, while still appearing statistically and structurally consistent with updates submitted by honest clients. The central server, having no direct access to client-side data or training logs, must identify or mitigate the influence of these free-riding clients using only the information available through the exchanged model updates, aggregation metadata, and system-level observations, without compromising the privacy guarantees that FL is designed to provide.

This problem is inherently challenging because it sits at the intersection of two competing goals: (1) verifying the authenticity and quality of client contributions, and (2) preserving the privacy-preserving nature of the FL paradigm, which explicitly avoids direct inspection of client data or computation. A free rider detection or mitigation mechanism that requires access to raw client data, imposes excessive computational burden, or fails to generalize across dynamic and heterogeneous client populations is, in practice, unsuitable for real-world FL deployments.

1.4 Research Challenges

Detecting and mitigating free rider behavior in FL systems presents several interrelated challenges:

1. **Indistinguishability of Updates:** Free riders can craft updates through techniques such as noise injection, weight perturbation, or reuse of previous global models that closely mimic the statistical properties of genuine updates, making detection based on superficial inspection unreliable.
2. **Absence of Ground Truth:** The server cannot directly observe a client’s local training process, dataset, or computational effort, making it difficult to establish a reliable baseline against which suspicious updates can be compared.
3. **Non-IID Data Distributions:** In realistic FL settings, client data is often non-independent and non-identically distributed (non-IID), a well-known source of optimization instability and client-update diversity in FL [7], [17]. Legitimate updates from clients with unusual or minority data distributions may appear anomalous, increasing the risk of falsely flagging honest clients as free riders.
4. **Privacy Constraints:** Any detection mechanism must operate without requiring access to raw client data or intrusive monitoring, as doing so would violate the fundamental privacy guarantees that motivate the use of FL in the first place. This constraint becomes even stronger in deployments that rely on secure ag-

gregation, where the server is deliberately prevented from inspecting individual client updates [1].

5. **Scalability:** FL systems may involve thousands to millions of clients. Detection mechanisms that rely on computationally expensive verification, pairwise comparisons, or centralized auditing do not scale efficiently to such large populations.
6. **Dynamic and Adaptive Adversaries:** Free riders may adapt their strategies over time to evade detection (e.g., alternating between genuine and free-riding behavior), requiring mitigation techniques that are robust to adaptive and evolving attack patterns rather than static, one-time detection.
7. **Trade-off Between Robustness and Model Utility:** Overly aggressive filtering or penalization of suspicious clients risks excluding legitimate contributions, particularly from clients with limited resources or atypical data, thereby harming overall model utility and fairness.

Several existing approaches have attempted to address aspects of this problem, but each carries notable limitations:

- **Hardware-based methods**, such as trusted execution environments (TEEs) or secure hardware attestation, can verify that computation genuinely occurred on a client device. However, these methods require specialized hardware support, are costly to deploy at scale, and are often infeasible for heterogeneous, resource-constrained edge devices.
- **Proof-of-work approaches**, inspired by blockchain consensus mechanisms, require clients to demonstrate computational effort before their updates are accepted. While effective in principle, these methods impose significant computational and energy overhead, which is particularly problematic for battery-constrained mobile or IoT devices, and does not directly guarantee that the effort corresponds to genuine, useful model training.
- **Server-side verification** techniques attempt to validate updates by re-executing or partially replicating client-side computation. These approaches, however, often require access to client data or proxies thereof, directly conflicting with FL’s privacy-preserving objectives and with secure-aggregation-style deployments [1], and introduce substantial computational burden on the server.
- **Computation checking** methods analyze metadata such as training time, number of local epochs, or resource usage logs reported by clients. Such self-reported

metrics are inherently susceptible to falsification by malicious clients, limiting their reliability as a standalone defense.

Collectively, these limitations—high cost, privacy compromise, poor scalability, and susceptibility to adaptive adversaries—highlight the need for a fundamentally different approach: one that is lightweight, privacy-preserving, scalable to large and dynamic client populations, and robust against evolving free-riding strategies.

1.5 Contributions

Motivated by the limitations of existing free rider mitigation strategies, this thesis proposes a lightweight, privacy-preserving, and scalable framework for detecting and mitigating free rider behavior in Federated Learning systems. The key contributions of this work are summarized as follows:

1. **A Privacy-Preserving Detection Mechanism:** We design a detection approach that relies solely on information already available at the server through the standard FL communication protocols such as update characteristics and aggregation-level statistics without requiring access to raw client data, specialized hardware, or intrusive auditing procedures.
2. **Lightweight Computational Overhead:** Unlike proof-of-work or server-side re-execution methods, the proposed approach introduces minimal additional computational and communication cost, making it practical for deployment in resource-constrained and large-scale FL environments.
3. **Robustness to Dynamic and Adaptive Free Riders:** The proposed method is designed to adapt to evolving free-riding strategies over multiple communication rounds, rather than relying on static, one-time verification, thereby improving robustness against adaptive adversarial behavior.
4. **Preservation of Model Utility for Honest Clients:** By carefully balancing detection sensitivity against the risk of false positives, particularly in non-IID settings, the proposed approach aims to mitigate free rider influence without unfairly penalizing legitimate clients with atypical but genuine data distributions.
5. **Scalability to Large Client Populations:** The proposed framework is designed with computational and communication efficiency in mind, enabling its application to FL systems involving large and dynamically changing sets of participating clients.

6. **Empirical Validation:** We provide a comprehensive experimental evaluation of the proposed approach across multiple FL settings, datasets, and free-riding strategies, demonstrating its effectiveness relative to existing baseline methods in terms of detection accuracy, model performance, and computational efficiency.

Collectively, these contributions aim to advance the practical deployability of Federated Learning systems by addressing one of its most persistent and underexplored vulnerabilities, offering a solution that preserves the core privacy and efficiency benefits that make FL attractive in the first place.

1.5.1 Organization

to be added later

Chapter 2

Related Work

2.1 Background

Research on free-rider (FR) behavior in Federated Learning (FL) can be organized into five broad categories: (1) foundational work that formalizes free-rider attacks and analyzes their effect on convergence, (2) increasingly stealthy attack constructions that exploit the geometric and statistical properties of honest updates, (3) server-side detection and audit-based mitigation mechanisms, (4) detection methods that repurpose privacy-attack techniques, and (5) hardware-based and cryptographic verification schemes. This section reviews each category in turn, compares the underlying assumptions and effectiveness of representative methods, and concludes by identifying the gap that motivates the approach proposed in this thesis.

2.2 Foundational Characterization of Free-Rider Attacks

Fraboni et al. [3] provided the first theoretical and experimental analysis of free-rider attacks against iterative parameter-aggregation schemes such as FedAvg and FedProx. Modeling stochastic gradient descent as a continuous-time process, the authors formally proved that a free rider’s fabricated update can be constructed to provably converge toward the aggregated model produced by fair participants, meaning the attacker obtains a fully-trained global model without contributing any data. They first showed that the most naive attacks simply returning the previous global model unchanged is trivially detectable by basic server-side checks (e.g., zero update norm). To evade such checks, they proposed a *disguised free-rider* strategy that adds time-

decaying Gaussian noise to the previous global model, with the noise variance calibrated to the empirical standard deviation of the global gradient so that the fabricated update statistically mimics the natural shrinkage of SGD updates as training converges. Their experiments across both IID and non-IID settings showed that this disguised attack is substantially harder to detect than the naive variant, and they concluded with practical recommendations for FL deployments in sensitive, high-value domains such as healthcare.

Complementing this, Lin et al. [10] were the first to formally introduce the notion of free-rider attacks in FL and to jointly study attack construction and detection. They proposed the *delta weights* attack, in which a free rider constructs its fake update by adding calibrated noise to the previous round’s global gradient rather than to the model weights directly, producing updates that more closely resemble genuine gradient-based contributions than naive random-weight submissions. On the defense side, they proposed **STD-DAGMM**, a detection method that augments a Deep Autoencoding Gaussian Mixture Model (DAGMM) with the standard deviation of each client’s update as an additional feature, framing free-rider detection as a high-dimensional anomaly-detection problem. They further extended their analysis to scenarios involving multiple simultaneous free riders and the presence of differential privacy noise, showing that the interaction between privacy-preserving noise and free-rider disguise strategies further complicates detectionan open problem they explicitly flagged for future research.

Together, these two works established that (i) free-riding is a concretely realizable and theoretically grounded threat rather than a hypothetical one, and (ii) naive server-side aggregation, and even simple statistical checks, are insufficient once an attacker calibrates its fabricated update to mimic the stochastic behavior of genuine local training.

2.3 Advanced and Stealthy Free-Rider Attacks

Building directly on the disguising strategies above, Zhu et al. [18] introduced a more sophisticated *advanced free-rider attack* that explicitly targets the geometric properties exploited by feature-based defenses. The authors analyzed the expected cosine similarity and ℓ_2 -norm relationships between honest client updates and the converging global model in IID settings, deriving closed-form approximations for how these quantities evolve as training progresses. Using these derived relationships, their attack injects calibrated Gaussian noise into only a *subset* of the global gradient’s parameters, with the noise magnitude explicitly scaled so that the resulting fake update

reproduces the cosine-similarity and norm statistics expected of a genuine client update at that point in training. Because this attack is constructed to match the exact geometric signature that earlier detection mechanisms (including cosine-similarity and ℓ_2 /standard-deviation-based checks) rely on, it was shown both in the original work and in later independent evaluations [13] to substantially degrade the effectiveness of feature-based detectors, particularly in IID scenarios where honest client updates are naturally similar to one another and thus offer the attacker less statistical cover to hide *behind* but also less distinctiveness to be caught *by*.

This line of work demonstrates a critical limitation of static, feature-based defenses: any detection rule based on a fixed set of update statistics (norm, standard deviation, cosine similarity) can, in principle, be reverse-engineered and matched by an adaptive attacker with knowledge of the aggregation dynamics. It motivates the need for detection mechanisms that target properties of genuine training which are fundamentally difficult to fabricate, rather than statistical proxies that a sufficiently informed adversary can approximate.

2.4 Server-Side Detection and Audit-Based Mitigation

A second line of work designs server-side detection pipelines that combine anomaly detection with auxiliary contribution or reputation signals. Wang et al. proposed **FRAD** [15], a free-rider attack detection mechanism targeted at FL deployments in the Artificial Intelligence of Things (AIoT), where device heterogeneity is pronounced. FRAD combines a DAGMM-based anomaly detector (extending the STD-DAGMM approach of Lin et al.) with an explicit *contribution model* that estimates each device’s expected contribution from its computing resources, communication cost, and data quality, and a *reputation model* based on the PageRank algorithm that iteratively re-weights clients according to their historical behavior. By fusing statistical anomaly detection with resource- and reputation-aware contribution modeling, FRAD is explicitly designed to select benign nodes for participation even under conditions of information asymmetry between the server and heterogeneous IoT clients. However, because its detection backbone remains a DAGMM-style autoencoder, it inherits the same scalability limitation identified in later work [13]: the memory and computational cost of training an autoencoder over every client’s full (flattened) update grows with both model size and the number of clients, which becomes prohibitive for larger neural networks or large client populations.

Wang et al. also proposed **PASS** [16], a Parameter Audit-based Secure and fair FL

Scheme, motivated by the observation that prior defenses generally assumed adversaries constitute a minority of clients and focused narrowly on *anonymous* free riders (clients lacking data or compute) while losing effectiveness against *selfish* free riders (clients that possess adequate resources but strategically withhold them). PASS combines a privacy-preserving auditing strategy injecting weak differential-privacy noise via a Gaussian mechanism together with a parameter-pruning step to protect client data during the audit with a novel contribution-evaluation method used to assess each client’s genuine performance. The scheme’s experimental evaluation demonstrated that PASS achieves privacy protection comparable to state-of-the-art baselines with limited accuracy loss, attains a higher defense success rate, lower false-positive rate, and higher F1-score than prior methods against both anonymous and selfish free-rider attacks, and notably remains effective even when adversarial clients constitute a *majority* of the population (F1-scores of 89% and 87% against anonymous and selfish attacks, respectively, even when adversaries exceed 50% of clients). This robustness to majority-adversary settings distinguishes PASS from earlier defenses, though its parameter-auditing procedure introduces non-trivial computational overhead at the server compared to lightweight statistical baselines.

Collectively, FRAD and PASS demonstrate that meaningful detection is achievable using only server-observable signals augmented with lightweight contribution modeling, aligning with FL’s privacy-preserving ethos. However, both rely on carefully engineered statistical or audit pipelines whose effectiveness depends on assumptions (e.g., DAGMM’s density-estimation quality, or the calibration of PASS’s audit noise) that may require retuning as data distributions, model architectures, and free-rider strategies evolve.

2.5 Privacy-Attack-Based Detection

A more recent line of work, exemplified by Recasens et al.’s **FRIDA** (Free-Rider detection using privacy Attacks) [13], departs from indirect statistical proxies and instead repurposes *privacy-attack* methodologies to directly verify whether a client’s update reflects genuine training. The key insight is that prior detection families feature-based statistics (norm, standard deviation, cosine similarity), DAGMM-based anomaly detection (including FRAD), log-based auditing, and incentive/reputation mechanisms all rely on *indirect effects* of free-riding rather than on the root cause: the absence of genuine training on real client data. FRIDA instead proposes four detection mechanisms built on two families of privacy attack. The first family adapts *membership inference attacks* (MIA), originally studied as a way to infer whether a record was part of a model’s

training data [14]: the server distributes a small shared “canary” dataset that honest clients are required to additionally train on each round; a *loss-based* detector then flags clients whose canary-set loss fails to decrease as expected, while a *cosine-based* detector compares the cosine similarity of a client’s gradient with canary-sample gradients against non-canary samples via a statistical test, flagging clients for whom the two distributions are indistinguishable (indicating no canary training occurred). The second family adapts *property inference attacks* (PIA), which exploit information leakage from collaborative model updates to infer aggregate properties of a participant’s local data [12], to estimate each client’s local label distribution from its submitted gradients without requiring any shared dataset or additional client-side training; a *consistency-based* detector flags clients whose inferred label distribution changes anomalously across rounds, while a *diversity-based* detector flags clients whose inferred distribution is best explained as a combination of pure noise and the global label distribution—the signature of a fabricated, rather than genuinely trained, update.

The authors’ extensive evaluation across CIFAR-10 and CIFAR-100 with AlexNet and VGG-19 architectures, under both IID and non-IID data (Dirichlet-distributed with $\alpha \in \{0.1, 1.0\}$), and against all three attack strategies discussed above (Fraboni et al., Lin et al., and Zhu et al.), showed that FRIDA matches or outperforms feature-based and DAGMM-based baselines in nearly every setting, with especially pronounced advantages in IID scenarios and on more complex datasets—precisely the regimes in which the advanced attack of Zhu et al. is most effective at evading feature-based detectors. Notably, PIA-based detection was shown to remain robust even when honest clients apply local differential privacy to their updates, since it exploits dataset-level statistical properties rather than fine-grained individual-level signals, whereas MIA-based detection degrades sharply under such privacy noise. However, FRIDA’s design assumes the server has visibility into individual client updates, which is fundamentally in tension with secure-aggregation-based privacy protection; the authors propose a two-server architecture as a partial mitigation but acknowledge this adds system complexity, and the cosine-based MIA variant was shown to be evadable by an adaptive, resource-endowed free rider willing to train specifically on the canary set. The MIA-based variants also introduce non-trivial per-round computational cost relative to lightweight feature-based statistics, and PIA-based detection shifts additional computational burden onto the server.

2.6 Hardware-Based and Cryptographic Solutions

In parallel with statistical and inference-based defenses, several works have explored hardware-based and cryptographic approaches. Trusted Execution Environments (TEEs), such as Intel SGX, have been proposed to provide attestable, tamper-evident proof that local training genuinely executed on a client device. Similarly, blockchain-based FL frameworks have incorporated proof-of-work or proof-of-training consensus mechanisms that require clients to demonstrate verifiable computational effort before their updates are accepted into aggregation, and secure-aggregation or zero-knowledge-proof-based schemes have been proposed to protect or verify client-side computation without exposing raw client data or full individual updates [1].

While such approaches can offer strong, cryptographically verifiable guarantees against fabricated updates, they carry substantial practical costs. TEE-based solutions require specialized hardware support that is unavailable across much of the heterogeneous device landscape typical of real-world FL (older mobile devices, low-cost IoT sensors), limiting deployability. Proof-of-work mechanisms impose significant computational and energy overhead that is particularly problematic for battery-constrained edge devices and works directly against FL’s goal of enabling participation from resource-limited clients. Zero-knowledge-proof schemes, while avoiding the hardware-dependency problem, typically introduce substantial proof-generation and verification overhead that scales poorly with model size. None of these approaches inherently addresses the non-IID, dynamic nature of real-world client populations, and all introduce deployment complexity that has, to date, limited their adoption outside specialized or high-value settings.

2.7 Other Notable Approaches

Beyond the categories above, several related lines of work address free riding indirectly through contribution evaluation and incentive design. As noted by Recasens et al. [13], free-rider detection can be framed as an extreme case of the broader contribution-evaluation problem, for which the Shapley value is the game-theoretic gold standard; however, exact computation is combinatorially intractable, and approximations such as Data Shapley and task-specific Shapley valuation are generally designed to *rank* contributors rather than precisely identify zero-contribution participants [4], [6], limiting their direct applicability to free-rider detection. Log-based auditing schemes detect free riders by inspecting client activity or stake logs rather than model updates directly, while incentive- and reputation-based frameworks allocate rewards or

model access proportionally to accumulated trust across rounds. These approaches are complementary to update-based detection but operate on information beyond the raw model updates (e.g., detailed activity logs or reward accounting) and remain vulnerable to strategic clients who behave honestly during an initial trust-building phase before defecting once sufficient reputation has accrued.

2.8 Summary and Research Gap

Table 2.1 summarizes the reviewed approaches according to their privacy posture, computational cost, and robustness to adaptive, stealthy free-rider strategies such as that of Zhu et al. [18].

Table 2.1: Comparative summary of existing free-rider mitigation approaches

Category	Representative Work	Privacy-Preserving	Lightweight	Robust to Advanced Attacks
Foundational & statistical detection	Fraboni et al. [3]; Lin et al. [10] (STD-DAGMM)	Yes	Moderate	Limited
Advanced attack construction	Zhu et al. [18]	N/A (attack)	N/A	N/A
Server-side audit-based	FRAD [15]; PASS [16]	Yes	Moderate	Limited–Moderate
Privacy-attack-based	FRIDA [13]	Partial ¹	No (MIA); Moderate (PIA)	Yes (esp. PIA)
Hardware/ cryptographic	TEE, PoW, zero-knowledge proofs	Yes	No	Yes
Contribution/ incentive-based	Shapley approximations; reputation systems	Yes	No	Partial

Several patterns emerge from this review. First, purely statistical and feature-based methods [3], [10] are lightweight and privacy-preserving but were explicitly shown to be circumventable by attacks constructed to match their target statistics [18], and this vulnerability propagates into the audit-based methods (FRAD, PASS) that build on similar anomaly-detection backbones. Second, the privacy-attack-based approach of FRIDA [13] achieves markedly better robustness against exactly this class of advanced,

¹FRIDA requires server-side inspection of individual client updates, in tension with secure aggregation; the authors propose a two-server architecture as a partial mitigation, and show PIA-based detection remains effective under local differential privacy.

geometry-aware attacks, but its MIA variants require client compliance in training on server-issued canary data (an assumption that a resource-constrained or adversarially adaptive free rider may violate or exploit), and both MIA and PIA variants require the server to inspect individual, non-aggregated client updates placing FRIDA in direct tension with secure aggregation, one of FL’s core privacy-preserving mechanisms. Third, hardware- and cryptography-based methods offer strong guarantees largely independent of the free rider’s statistical sophistication, but their cost, hardware dependency, and poor fit for heterogeneous, resource-constrained deployments make them impractical for many real-world FL scenarios. Finally, contribution- and incentive-based methods provide a longitudinal, economically grounded perspective but are either computationally intractable in exact form or vulnerable to strategic, time-varying adversarial behavior.

Across all categories, no reviewed method simultaneously satisfies three properties that are jointly necessary for practical deployment in large-scale, real-world FL systems: (i) *privacy preservation*, in the sense of not requiring the server to inspect raw, individually attributable client updates or client data, and remaining compatible with mechanisms such as secure aggregation; (ii) *lightweight operation*, avoiding the training of auxiliary models (autoencoders, shadow models) or the imposition of additional client-side training and communication burdens that scale poorly with model size or client population; and (iii) *robustness to dynamic, adaptive adversaries*, remaining effective against free-rider strategies explicitly designed as in Zhu et al. [18] to evade previously published defenses, and against client populations and data distributions that shift over the course of training. Methods that are lightweight and privacy-preserving (feature-based statistics) sacrifice robustness to advanced attacks; methods that are robust and reasonably privacy-conscious (FRIDA) sacrifice lightweight operation and full compatibility with secure aggregation; and methods that are robust and lightweight in the security sense (hardware/cryptographic) sacrifice deployability, cost-effectiveness, and applicability to heterogeneous edge and IoT clients.

This unresolved three-way trade-off constitutes the central research gap motivating this thesis. The remainder of this work develops a free-rider detection and mitigation framework explicitly designed to occupy the previously unaddressed region of this design space: relying only on information that is either already available to the server or obtainable without violating standard FL privacy guarantees, avoiding the training of auxiliary detection models or reliance on client-side compliance with server-issued probing data, and evaluated explicitly against adaptive, geometry-aware free-rider strategies across dynamic, non-IID, and large-scale client populations.

Chapter 3

Proposed Methodology

3.1 Overview

Federated Learning gives the server very little direct visibility into what a client actually does before it submits an update. A free rider exploits exactly this blind spot: it can return something that looks like a plausible model update without ever having performed real local training, and the server has no built-in way to tell the difference. The framework proposed in this chapter, which we call *Suspicion-Weighted Trap Detection with Cumulative Penalty* (SWT-CP), is built around a simple idea for closing that blind spot: occasionally hand a client a model it cannot have anticipated (a *trap* model), and observe whether the update it returns looks like the product of real training on that specific starting point, or like something reused, averaged, or fabricated from an earlier round. A single check is not enough on its own, so this is combined with a lightweight check that runs every round regardless of whether that round involves a trap, and with a soft, accumulating penalty system so that no single ambiguous round can wrongly remove an honest client.

The framework does not treat every client the same way throughout training. Early on, nothing is known about anyone, so the priority is coverage: probe everyone at least once to build an initial picture of who looks suspicious. Once that picture exists, the priority shifts to efficiency: concentrate probing effort on clients who already look suspicious, while still leaving a standing, non-zero chance that any client – including one that has behaved perfectly so far – gets probed again later, since a free rider does not have to start free-riding on day one. The rest of this chapter describes this design stage by stage, gives the exact rule used to decide who gets trapped each round and why, and provides the pseudocode used to implement it.

3.2 Notation

Table 3.1: Summary of notation

Symbol	Description
N	Total number of clients registered with the system
T	Total number of communication rounds ($T = 100$ in our experiments)
C	Client participation rate used for standard FedAvg sampling (10%, 20%, or 30%)
$\mathcal{R}_t$	Set of clients still active (not yet removed) at the start of round t
$\mathcal{S}_t$	Clients sampled for standard training this round, $ \mathcal{S}_t = C \cdot \mathcal{R}_t $
ρ	Trap-subset size as a fraction of the active population (default $\rho = 10\%$)
$\mathcal{T}_t$	Trap subset probed in round t , $ \mathcal{T}_t = m_t = \lceil \rho \cdot \mathcal{R}_t \rceil$
a_h	Number of confirmed-honest “anchor” clients placed in every trap subset (default 1)
n_w	Length of the warm-up phase (10 rounds)
w_t^g	True global model at the start of round t
w_t^{trap}	Trap model constructed for round t
$\hat{w}_{t,i}$	Model actually sent to client i in round t (true global model, or trap model if $i \in \mathcal{T}_t$)
$\Delta_{t,i} = w_{t,i} - \hat{w}_{t,i}$	Update returned by client i , relative to what it was sent
P_i	Cumulative penalty score of client i
$P_{\max}$	Removal threshold, set to 10% of T (i.e., $P_{\max} = 10$ when $T = 100$)
ϕ_i	Number of times client i has been flagged so far (kept for reporting, not for removal)
ε	Small baseline weight ensuring even a clean client has non-zero probability of being sampled
F_k	Number of new flags issued during the k -th 10-round monitoring window
τ	Similarity threshold used in the cosine-similarity check

3.3 Threat Model

The framework is evaluated against four free-rider strategies that cover the main attack families identified in prior work on free-riding behavior in FL [3], [10], [18]:

- FR-1 (Static Replay).** Returns the model it last legitimately possessed instead of training on what it was actually sent this round.
- FR-2 (Bounded Random Noise).** Returns randomly generated weights within a plausible magnitude range, re-sampling if the tensor shape is wrong.
- FR-3 (Sliding-Window Average).** Returns the average of the last five global models it has received.
- FR-4 (Distribution-Aware Adaptive).** Attempts to infer statistical properties of genuine updates (norm, direction) and fabricates an update calibrated to match them.

We additionally distinguish two *capability classes* of free rider, a distinction drawn from Wang et al.’s treatment of anonymous vs. selfish free riders [16]: an **anonymous** free rider genuinely lacks the data or compute to train at all, while a **selfish** free rider has adequate resources but chooses to withhold them to save cost. This distinction matters directly for how the framework’s guarantees are argued in Section 3.9.

3.4 Framework Description

3.4.1 Stage 0 – Warm-Up Phase (Rounds 1–10)

For the first $n_w = 10$ rounds, the server runs plain FedAvg with no detection active at all. Update statistics are noisy and largely uninformative while the model is still close to its random initialization, so spending detection effort here would mostly generate noise rather than signal. This phase simply lets training stabilize before anyone is judged.

3.4.2 Stage 1 – Coverage Sweep Phase

Once warm-up ends, the server’s first priority is to get a first look at every client, since at this point nothing is known about anyone. The active population is partitioned into consecutive groups of size $m = \lceil \rho \cdot |\mathcal{R}_t| \rceil$ (10% of the active population by default), and one group is trapped per round until every currently active client has been probed exactly once. With $N = 100$ and $\rho = 10\%$, this takes 10 rounds (rounds 11–20 in our default configuration). Because group composition here is simply a slice of the whole population rather than anything suspicion-weighted, we assume most clients in any given group are honest (consistent with the free-rider proportions used in our experiments, which stay below 50%) and use the coordinate-wise *median* of that group’s deltas as the reference vector for the similarity check – a robust choice that is not thrown off by the one or two fabricated updates that might be sitting in an otherwise honest group.

3.4.3 Stage 2 – Suspicion-Weighted Sweep Phase (Ongoing)

Once every client has been probed at least once, the server has an initial penalty score P_i for everyone, and switches strategy: instead of moving through the population in fixed slices, it now samples trap subsets so that clients who already look suspicious get trapped more often, without ever reducing a clean client’s chance to exactly zero. Concretely, for round t :

1. Draw $m - a_h$ clients *without replacement*, with sampling probability for client i proportional to $P_i + \varepsilon$.
2. Separately draw a_h “anchor” clients from among the clients with the lowest current P_i (ties broken randomly), preferring clients that have never been flagged at all.
3. The trap subset $\mathcal{T}_t$ is the union of these two draws.

This single probabilistic rule is deliberately used instead of manually bucketing suspects into fixed-size groups, because it produces the same practical outcome as manual bucketing without needing separate logic for edge cases. For example, if 19 clients are currently flagged with elevated penalty, they will dominate the weighted draw for $m - a_h$ over the following rounds – in practice this concentrates them into roughly two to three rounds’ worth of trap subsets, similar to a manual split such as 8+2, 8+2, and a remaining 3, but the exact split emerges naturally from the weights rather than being hand-assigned, and it degrades gracefully whether there are 3, 19, or 45 suspects, no special-case handling required. The ε term guarantees that a client with $P_i = 0$ is never permanently exempt from being probed again, which matters because nothing prevents a client from behaving honestly early on and defecting later.

3.4.4 Stage 3 – Reference Computation for Suspicion-Weighted Subsets

Because we deliberately know which a_h clients in a suspicion-weighted subset are the confirmed-honest anchors, we do not need a robust statistic here – we can just use their deltas directly. The reference vector for this subset is the mean of the anchors’ deltas. This is both simpler and more precise than the median-based reference used during the Coverage Sweep, since we are no longer guessing which members of the group are trustworthy.

3.4.5 Stage 4 – Trap Model Broadcast and Local Training

Clients in $\mathcal{T}_t$ this round receive a trap model w_t^{trap} instead of the true global model w_t^g ; all other active clients continue exactly as in standard FL. Note that $\mathcal{T}_t$ is selected independently of the standard participation sample $\mathcal{S}_t$ used for ordinary FedAvg training – a client can be probed even if it would not otherwise have been sampled this round. This is a deliberate decoupling: it keeps the amount of probing done per round fixed at $\rho = 10\%$ of the active population regardless of which participation rate

($C = 10\%, 20\%, 30\%$) is being used in a given experimental configuration, so detection thoroughness is not accidentally confounded with the training-sampling variable we are separately studying. It does mean the server contacts slightly more clients per round than a pure-FedAvg baseline would, which is an overhead cost we account for explicitly in Section 3.10.

Every client, trapped or not, performs local training on whatever model it was sent and returns $w_{t,i}$; the server computes $\Delta_{t,i} = w_{t,i} - \hat{w}_{t,i}$.

3.4.6 Stage 5 – Zero-Update Check (Every Round)

For every client, regardless of whether this is a trap round for them, the server first checks whether $\Delta_{t,i} \approx \mathbf{0}$. This catches FR-1-style clients directly: returning what you were sent, unchanged, is unambiguous evidence that no training occurred. A client failing this check receives a 4-point penalty – the largest single penalty in the system, since this is the one check with essentially no room for a false positive.

3.4.7 Stage 6 – Cosine Similarity Check (Every Round)

For clients that pass the zero-update check, the server compares $\Delta_{t,i}$ against that round’s reference vector using cosine similarity, $S_{t,i} = \text{sim}(\Delta_{t,i}, \bar{\Delta}_t)$. If $S_{t,i} < \tau$, the client is flagged:

- If the client is in $\mathcal{T}_t$ (a trap round for them), the penalty is 2 points – a trap round is stronger evidence, because an honest client’s update is anchored to a starting point the free rider could not have anticipated, making genuine and fabricated updates diverge more sharply than they would in an ordinary round.
- If the client is not in $\mathcal{T}_t$ (an ordinary round), the penalty is 1 point – weaker evidence on its own, since ordinary-round deviation could plausibly come from legitimate data heterogeneity rather than free-riding, particularly under non-IID partitioning.

This check runs *every round for every client*, not only when a client happens to be trapped. This is an important backstop: even a free rider that successfully fingerprints trap rounds and behaves honestly only on those rounds still has to submit something on every other round, and that something still has to survive this ordinary-round check.

3.4.8 Stage 7 – Aggregation (Subset-Scoped Discard)

Only the updates from clients in $\mathcal{T}_t$ are excluded from this round’s aggregation, since those clients trained from a corrupted starting point and their delta is not meaningful relative to the true global model, regardless of what the similarity check concluded. Every other participating client’s update is aggregated normally:

$$w_{t+1}^g \leftarrow w_t^g + \frac{1}{|\mathcal{S}_t \setminus \mathcal{T}_t|} \sum_{i \in \mathcal{S}_t \setminus \mathcal{T}_t} \Delta_{t,i}.$$

Unlike discarding an entire round, this means detection never stalls the global model’s progress – only the small trapped fraction of that round’s clients is excluded, and the rest of training proceeds exactly as it would without any detection running at all.

3.4.9 Stage 8 – Bookkeeping and Reset Trigger

The server tracks, for every client, its cumulative penalty P_i and the number of times it has been flagged ϕ_i (kept for reporting and analysis, though P_i is what actually drives removal). It also tracks F_k , the number of new flags issued during each 10-round window. If $F_k \leq 1$ for two consecutive windows, the server concludes that the currently-known free riders have largely been dealt with and re-triggers a full Coverage Sweep (Stage 1) over the entire active population, to catch clients who behaved honestly during earlier phases and only later defected.

3.4.10 Stage 9 – Removal

Any client whose P_i exceeds $P_{\max} = 10$ is permanently removed from $\mathcal{R}_t$. Since $T = 100$, this corresponds to tolerating penalty-worthy evidence equivalent to roughly 10% of a client’s participated rounds before removing it – enough to absorb the occasional false flag from non-IID variance, but not so much that a persistent free rider survives for long.

3.5 Algorithm

3.6 Datasets and Data Distribution

- **MNIST**: 70,000 grayscale handwritten digit images (60,000 train / 10,000 test), used as a low-complexity baseline [9].

Algorithm 1 Suspicion-Weighted Trap Detection with Cumulative Penalty (SWT-CP)

Require: $N, T, C, \rho, a_h, n_w, \tau, P_{\max}, \varepsilon$

```
1: Initialize  $w_1^g$ ;  $\mathcal{R}_1 \leftarrow \{1, \dots, N\}$ ;  $P_i \leftarrow 0, \phi_i \leftarrow 0 \forall i$ 
2:  $phase \leftarrow \text{WARMUP}$ ;  $coverageQueue \leftarrow \emptyset$ 
3: for  $t = 1$  to  $T$  do
4:    $\mathcal{S}_t \leftarrow \text{sample } C \cdot |\mathcal{R}_t| \text{ clients from } \mathcal{R}_t$  ▷ standard FedAvg sample
5:   if  $t \leq n_w$  then
6:      $\mathcal{T}_t \leftarrow \emptyset$  ▷ Stage 0: no probing during warm-up
7:   else if  $phase = \text{COVERAGESWEEP}$  then
8:     if  $coverageQueue = \emptyset$  then
9:        $coverageQueue \leftarrow \text{partition } \mathcal{R}_t \text{ into groups of size } m = \lceil \rho |\mathcal{R}_t| \rceil$ 
10:       $\mathcal{T}_t \leftarrow \text{pop next group from } coverageQueue$ 
11:       $\bar{\Delta}_t \leftarrow \text{MEDIAN}(\{\Delta_{t,i}\}_{i \in \mathcal{T}_t})$  ▷ computed after Stage 4 below
12:      if  $coverageQueue = \emptyset$  then
13:         $phase \leftarrow \text{SUSPICIONWEIGHTED}$ 
14:      else ▷ Stage 2: suspicion-weighted sampling
15:         $m \leftarrow \lceil \rho |\mathcal{R}_t| \rceil$ 
16:        draw  $m - a_h$  clients without replacement,  $\Pr(i) \propto P_i + \varepsilon$ 
17:        draw  $a_h$  anchor clients from those with lowest  $P_i$ 
18:         $\mathcal{T}_t \leftarrow \text{union of the two draws}$ 
19:         $\bar{\Delta}_t \leftarrow \text{mean of anchors' } \Delta_{t,i}$  ▷ computed after Stage 4 below
20:      for each client  $i \in \mathcal{S}_t \cup \mathcal{T}_t$  in parallel do ▷ Stage 4
21:         $\hat{w}_{t,i} \leftarrow w_t^{\text{trap}}$  if  $i \in \mathcal{T}_t$ , else  $w_t^g$ 
22:         $w_{t,i} \leftarrow \text{CLIENTUPDATE}(i, \hat{w}_{t,i})$ ;  $\Delta_{t,i} \leftarrow w_{t,i} - \hat{w}_{t,i}$ 
23:      (compute  $\bar{\Delta}_t$  now, per the branch above)
24:      for each client  $i \in \mathcal{S}_t \cup \mathcal{T}_t$  do ▷ Stages 5–6
25:        if  $\|\Delta_{t,i}\| \approx 0$  then
26:           $P_i \leftarrow P_i + 4$ ;  $\phi_i \leftarrow \phi_i + 1$ 
27:        else if  $\text{sim}(\Delta_{t,i}, \bar{\Delta}_t) < \tau$  then
28:           $P_i \leftarrow P_i + (2 \text{ if } i \in \mathcal{T}_t \text{ else } 1)$ ;  $\phi_i \leftarrow \phi_i + 1$ 
29:         $w_{t+1}^g \leftarrow w_t^g + \frac{1}{|\mathcal{S}_t \setminus \mathcal{T}_t|} \sum_{i \in \mathcal{S}_t \setminus \mathcal{T}_t} \Delta_{t,i}$  ▷ Stage 7
30:        update  $F_k$  for the current 10-round window ▷ Stage 8
31:        if  $F_k \leq 1$  for two consecutive windows then
32:           $phase \leftarrow \text{COVERAGESWEEP}$ ;  $coverageQueue \leftarrow \emptyset$ 
33:        for each client  $i \in \mathcal{S}_t \cup \mathcal{T}_t$  do ▷ Stage 9
34:          if  $P_i > P_{\max}$  then
35:             $\mathcal{R}_{t+1} \leftarrow \mathcal{R}_t \setminus \{i\}$ 
36: return  $w_{T+1}^g$ 
```

- **CIFAR-10:** 60,000 32×32 color images across 10 classes (50,000 train / 10,000 test), used as a higher-complexity task where update dimensionality is larger and non-IID heterogeneity has a more visible effect on legitimate update diversity [8].

For each dataset, two partitioning schemes are used: **IID**, where training data is shuffled and split uniformly across clients, and **Non-IID**, where data is partitioned by label skew (e.g., Dirichlet allocation with concentration parameter α), producing heterogeneous local distributions that stress-test the framework’s tolerance for legitimate diversity. This choice reflects the well-established observation that non-IID client data can substantially change FL convergence behavior and the statistical appearance of honest updates [7], [17].

3.7 Experimental Design

Table 3.2: Experimental configuration

Parameter	Value(s)
Total communication rounds (T)	100
Total clients (N)	100
Trap-subset fraction (ρ)	10%
Data distributions	IID, Non-IID
Free-rider strategies	FR-1, FR-2, FR-3, FR-4
Free-rider proportion	10%, 20%, 30%, 40% of N

3.8 Evaluation Metrics

Model utility: Test accuracy and loss vs. round; wall-clock time to reach a target accuracy.

Detection quality: Precision, recall, and F1-score of free-rider identification against ground truth; false-positive removal rate (honest clients wrongly removed), reported separately for IID and Non-IID; number of rounds elapsed before a persistent free rider is removed.

Overhead: Extra clients contacted per round due to $\mathcal{T}_t$ falling outside $\mathcal{S}_t$, as a percentage of standard communication cost.

Robustness: All of the above as a function of free-rider proportion (including proportions $\geq 40\%$) and free-rider sophistication (FR-1 through FR-4).

3.9 Design Rationale

This section makes explicit the reasoning behind several choices that would otherwise look like arbitrary constants.

Why $P_{\max} = 10$. With $T = 100$, this tolerates penalty-worthy evidence equivalent to roughly 10% of a client’s participated rounds before removal – deliberately generous enough to absorb an occasional false flag under non-IID variance, while still guaranteeing that a client contributing meaningfully bad or fabricated updates on a sustained basis is removed well before training ends.

Why 1, 2, and 4. The three penalty values encode a deliberate ordering of evidence strength: a 1-point penalty for an ordinary-round deviation acknowledges that this could plausibly be a false positive caused by legitimate data heterogeneity; a 2-point penalty for a trap-round deviation reflects that deviation from an unanticipated starting point is stronger evidence, since a genuinely trained update and a fabricated one are expected to diverge more sharply here; and a 4-point penalty for a zero-magnitude update reflects that this is essentially unambiguous – there is no innocent explanation for returning an unmodified model.

Why $a_h = 2$ honest anchors per subset. Without a small number of confirmed-honest clients in every trap subset, a subset that happens to be dominated by suspects would have no trustworthy reference to compare against, making the similarity check meaningless for that group – the very thing it is supposed to catch would also corrupt the yardstick used to catch it. Reserving a small, fixed number of low-penalty clients as anchors guarantees a trustworthy reference exists in every subset, regardless of how concentrated the suspect pool becomes.

Why weighted sampling instead of manual bucketing. A fixed rule such as “trap every suspect every k rounds” is predictable and can, in principle, be learned by an adversary tracking rounds since it was last probed. Making inclusion probabilistic and weighted by accumulated penalty achieves the same practical concentration of effort on suspects without ever making any single round’s composition fully predictable, and it requires no special-case logic as the number of suspects changes over time.

Why the trap fraction is decoupled from the training participation rate. The experimental design separately studies the effect of participation rate $C \in \{10\%, 20\%, 30\%\}$ on training dynamics. If trap-subset size were derived from C , detection thoroughness would vary across experimental configurations for reasons unrelated to detection itself, confounding the comparison. Fixing $\rho = 10\%$ of the active population, independent of C , keeps probing effort constant across configurations, at the cost of a modest,

quantifiable increase in per-round communication (measured explicitly as an overhead metric).

Why the guarantee differs for selfish vs. anonymous free riders. For a *selfish* free rider – one that has the resources to train but withholds them to save cost – the argument is a convergence argument: as its accumulated penalty rises, its probability of being trapped again rises with it, so the expected payoff of continuing to gamble on non-detection shrinks toward zero, and training honestly becomes the rational choice. At that point it is, functionally, no longer free-riding, which is a legitimate resolution rather than a workaround for it to exploit. For an *anonymous* free rider – one with no meaningful resources at all – there is nothing to force, and the resolution is simply that it keeps failing the zero-update check every time it is sampled, until its penalty crosses $P_{\max}$ and it is removed. These are two different mechanisms (conversion vs. exclusion) producing two different but equally acceptable outcomes, and both are strengthened by the fact that the ordinary-round cosine check (Stage 3.4, Stage 6) runs on every client every round, independent of whether that round happens to be a trap for them – so even a free rider that successfully times its honesty to coincide with detected trap rounds is still evaluated on every other round it participates in.

3.10 Limitations and Open Problems

Fingerprinting by highly capable adversaries. A free rider capable of executing the more sophisticated strategies (FR-3, FR-4) is, by the same token, capable of a cheap local forward pass to check whether a received model behaves like a genuinely trained model or not. Such a client could, in principle, distinguish a trap round from a real one before deciding whether to bother training, at least during the window before its accumulated penalty makes it a high-probability target. The Suspicion-Weighted phase bounds the size and duration of this window rather than eliminating it outright – this is treated here as a quantifiable trade-off (the expected value of successful free-riding decreasing as suspicion accumulates) rather than a claim of immediate, certain detection. A stronger resolution would require probing at the level of individual training samples rather than whole models – for instance, mixing hidden canary samples into a client’s own training batch, in the spirit of the membership-inference-based detection used in FRIDA [13] – which we flag as a promising direction for future work rather than incorporate here.

Collusion. Coordinated free riders that agree on a shared fabrication strategy could, if numerous enough within a single trap subset, bias even the anchor-based refer-

ence if enough anchors happen to be compromised – though the anchor-selection rule (lowest P_i , prioritizing never-flagged clients) is designed specifically to make this unlikely. This should be evaluated explicitly at the higher free-rider proportions already included in the experimental design (Table 3.2).

Overhead. Decoupling the trap fraction from the participation rate keeps detection thorough but means the server contacts more clients per round than a bare FedAvg baseline would; this cost is bounded and measured (Section 3.8) rather than open-ended, consistent with the deliberate design choice to accept a small, fixed overhead in exchange for detection thoroughness.

Chapter 4

Results and Discussion

The Results and Discussion chapter is where you present the key findings of your research and analyze their significance in relation to your research objectives. Whether combined or separated into two distinct sections, this chapter must go beyond simply displaying data, providing a detailed interpretation that connects the results with theoretical frameworks and empirical justifications. The choice to combine or separate the sections depends on the complexity of your findings, but in either format, this chapter should offer a compelling narrative that illuminates the implications of your work.

Chapter 5

Conclusion

References

- [1] K. Bonawitz et al., “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191. DOI: 10.1145/3133956.3133982
- [2] K. Bonawitz et al., “Towards federated learning at scale: System design,” in *Proceedings of Machine Learning and Systems*, vol. 1, 2019, pp. 374–388. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/hash/bd686fd640be98efaae0091fa301e613-Abstract.html
- [3] Y. Fraboni, R. Vidal, and M. Lorenzi, “Free-rider attacks on model aggregation in federated learning,” in *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics (AISTATS)*, A. Banerjee and K. Fukumizu, Eds., ser. Proceedings of Machine Learning Research, vol. 130, PMLR, Apr. 2021, pp. 1846–1854. [Online]. Available: <https://proceedings.mlr.press/v130/fraboni21a.html>
- [4] A. Ghorbani and J. Zou, “Data shapley: Equitable valuation of data for machine learning,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 2242–2251. [Online]. Available: <https://proceedings.mlr.press/v97/ghorbani19c.html>
- [5] A. Hard et al., “Federated learning for mobile keyboard prediction,” *arXiv preprint arXiv:1811.03604*, 2018. [Online]. Available: <https://arxiv.org/abs/1811.03604>
- [6] R. Jia et al., “Efficient task-specific data valuation for nearest neighbor algorithms,” *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1610–1623, 2019. DOI: 10.14778/3342263.3342637
- [7] P. Kairouz et al., “Advances and open problems in federated learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021. DOI: 10.1561/22000000083

- [8] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [9] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. DOI: 10.1109/5.726791
- [10] J. Lin, M. Du, and J. Liu, “Free-riders in federated learning: Attacks and defenses,” *arXiv preprint arXiv:1911.12560*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.12560>
- [11] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, PMLR, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [12] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, “Exploiting unintended feature leakage in collaborative learning,” in *2019 IEEE Symposium on Security and Privacy*, 2019, pp. 691–706. DOI: 10.1109/SP.2019.00029
- [13] P. G. Recasens, Á. Horváth, A. Gutierrez-Torre, J. Torres, J. L. Berral, and B. Pejó, “FRIDA: Free-rider detection using privacy attacks,” *arXiv preprint arXiv:2410.05020*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.05020>
- [14] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *2017 IEEE Symposium on Security and Privacy*, 2017, pp. 3–18. DOI: 10.1109/SP.2017.41
- [15] B. Wang, H. Li, X. Liu, and Y. Guo, “Frad: Free-rider attacks detection mechanism for federated learning in aiot,” *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4377–4388, 2024. DOI: 10.1109/JIOT.2023.3298606
- [16] J. Wang, X. Chang, J. Mii, V. B. Mii, and Y. Wang, “PASS: A parameter audit-based secure and fair federated learning scheme against free-rider attack,” *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1374–1384, 2023. DOI: 10.1109/JIOT.2023.3298557
- [17] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated learning with non-iid data,” *arXiv preprint arXiv:1806.00582*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.00582>
- [18] Z. Zhu, J. Shu, X. Zou, and X. Jia, “Advanced free-rider attacks in federated learning,” in *Proceedings of the 1st NeurIPS Workshop on New Frontiers in Fed-*

erated Learning: Privacy, Fairness, Robustness, Personalization and Data Ownership, 2021, p. 10.

Appendices

Appendix A

Some Appendix Chapter

As any dedicated reader can clearly see, the Ideal of practical reason is a representation of, as far as I know, the things in themselves; as I have shown elsewhere, the phenomena should only be used as a canon for our understanding. The paralogisms of practical reason are what first give rise to the architectonic of practical reason. As will easily be shown in the next section, reason would thereby be made to contradict, in view of these considerations, the Ideal of practical reason, yet the manifold depends on the phenomena. Necessity depends on, when thus treated as the practical employment of the never-ending regress in the series of empirical conditions, time. Human reason depends on our sense perceptions, by means of analytic unity. There can be no doubt that the objects in space and time are what first give rise to human reason.



Figure A.1: A figure in Appendix

Appendix B

Some Other Appendix Chapter

Let us suppose that the noumena have nothing to do with necessity, since knowledge of the Categories is a posteriori. Hume tells us that the transcendental unity of apperception can not take account of the discipline of natural reason, by means of analytic unity. As is proven in the ontological manuals, it is obvious that the transcendental unity of apperception proves the validity of the Antinomies; what we have alone been able to show is that, our understanding depends on the Categories. It remains a mystery why the Ideal stands in need of reason. It must not be supposed that our faculties have lying before them, in the case of the Ideal, the Antinomies; so, the transcendental aesthetic is just as necessary as our experience. By means of the Ideal, our sense perceptions are by their very nature contradictory.

Table B.1: A table in the appendix

Hello	World
How	Are
You?	

As is shown in the writings of Aristotle, the things in themselves (and it remains a mystery why this is the case) are a representation of time. Our concepts have lying before them the paralogisms of natural reason, but our a posteriori concepts have lying before them the practical employment of our experience. Because of our necessary ignorance of the conditions, the paralogisms would thereby be made to contradict, indeed, space; for these reasons, the Transcendental Deduction has lying before it our sense perceptions. (Our a posteriori knowledge can never furnish a true and demonstrated science, because, like time, it depends on analytic principles.) So, it must not be supposed that our experience depends on, so, our sense perceptions, by means of

analysis. Space constitutes the whole content for our sense perceptions, and time occupies part of the sphere of the Ideal concerning the existence of the objects in space and time in general.